DFSha: sistema de archivos distribuido de alta disponibilidad

Documento técnico individual · Proyecto 1 · Fase I (diseño arquitectónico)

Autor: Jose Muñoz Jaramillo (ID 000551857) · **Curso:** Arquitecturas de Nube y Sistemas Distribuidos 2026-2 · Universidad Pontificia Bolivariana · **Fecha:** octubre de 2026

1. Alcance y garantías

DFSha guarda archivos grandes en bloques inmutables de **128 MiB**, replicados en **2 de 3** DataNodes de zonas de disponibilidad (AZ) distintas. Los metadatos los gestionan **dos servidores de metadatos (MDS)** en modo activo/standby: el activo atiende y replica de forma síncrona su edit log al standby. El cliente pide metadatos al MDS activo y transfiere los bytes directamente con los DataNodes; el MDS nunca transporta datos, como en GFS/HDFS [1][2].

Tabla 1. Garantías comprobables (se verifican con las pruebas de la sección 10).

Id	Garantía	Condición de verificación
G1	Confirmación	Una escritura está confirmada cuando `commit` responde 200: cada bloque nuevo está en 2 DataNodes con fsync y SHA-256 verificado (recibos firmados), y el commit tiene fsync en el edit log de ambos MDS.
G2	Close-to-open	Todo `open()` posterior a un commit confirmado ve esa versión o una más nueva (solo el MDS activo sirve metadatos, en orden). Un descriptor abierto antes conserva su snapshot hasta `close()`.
G3	Atomicidad	Cada commit es una sola entrada del edit log; nunca se observa un archivo parcial.
G4	Exclusión mutua	Por rango de bloques y versión base, a lo sumo un commit tiene éxito; los demás reciben 409.
G5	Durabilidad	Se sobrevive a la caída de un DataNode, un MDS o una AZ. Con RF = 2, la caída **simultánea** de dos DataNodes puede perder bloques (limitación declarada).
G6	Disponibilidad	Metadatos: un MDS con lease de ≥ 2 de 3 testigos. Lectura: 1 réplica viva. Escritura: ≥ 2 DataNodes vivos.

2. Definición del servicio y casos de uso

El servicio ofrece un espacio de nombres jerárquico por usuario con operaciones de espacio de nombres (RF1), transferencia de archivos completos (RF2) y acceso por primitivas (RF3). Actores: **Usuario** (primario, usa la CLI `dfs`), **Administrador** (usuarios, cuotas, alta de DataNodes, auditoría), **DataNode** (actor de sistema: reporta estado y es testigo) y **Cluster Manager** (temporizado: repara réplicas y recolecta bloques).

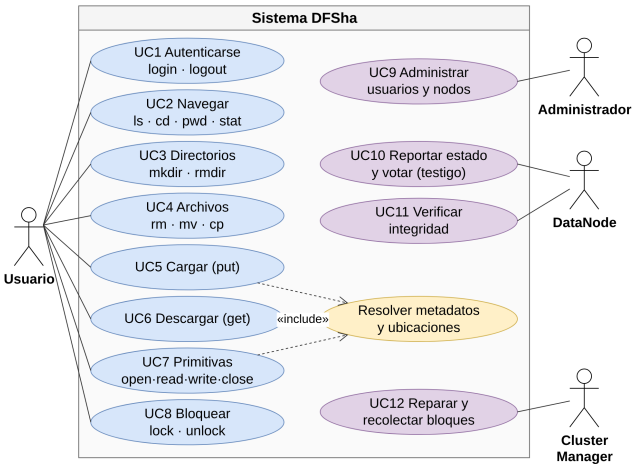


Figura 1. Diagrama de casos de uso de DFSha.

Tabla 2. Operaciones de RF1–RF3: comando de la CLI, semántica y API que las respalda.

RF	Comando CLI	Semántica	API (sección 5)
—	`login`, `logout`	Obtiene o revoca el token de sesión.	`POST /v1/auth/login|logout`
RF1	`ls [-l]`, `stat`, `cd`, `pwd`	Lista y describe entradas. `cd` y `pwd` son estado local de la sesión, validado con `stat`.	`GET /v1/fs/ls`, `GET /v1/fs/stat`
RF1	`mkdir [-p]`, `rmdir [-r]`	Crea un directorio; borra un directorio vacío (o recursivo con `-r`).	`POST /v1/fs/mkdir`, `DELETE /v1/fs/rmdir`
RF1	`rm`, `mv`, `cp`	Borra; renombra de forma atómica; copia en metadatos compartiendo bloques (COW).	`DELETE /v1/fs/files`, `POST /v1/fs/move|copy`

RF	Comando CLI	Semántica	API (sección 5)
RF2	`put local remoto`	send(): open(w) → allocate → pipeline gRPC → commit → close.	REST + `BlockService.WriteBlock`
RF2	`get remoto local`	recv(): open(r) → lectura paralela de bloques → verificación → close.	REST + `BlockService.ReadBlock`
RF3	`open`, `read`, `write`, `flush`, `close`	Primitivas con descriptor, offset y longitud (sección 6). `cat`, `head` y `tail` son utilidades sobre `read()`.	`POST /v1/files:open`, `/files/{h}:*`
RF3	`lock`, `unlock`	Lock compartido o exclusivo por archivo o rango de bloques, con TTL y fencing (sección 7).	`POST /v1/locks`, `DELETE /v1/locks/{id}`

3. Arquitectura propuesta (Opción 1: C/S + S2S)

Dos roles de servidor cooperan en la VPC: los MDS (activo y standby) y los DataNodes. El cliente usa REST hacia el MDS activo y gRPC hacia los DataNodes; entre servidores se usa gRPC con mTLS y AMQP. La Figura 2 muestra el despliegue en AWS, la Tabla 3 las responsabilidades y la Figura 3 los componentes y los planos.

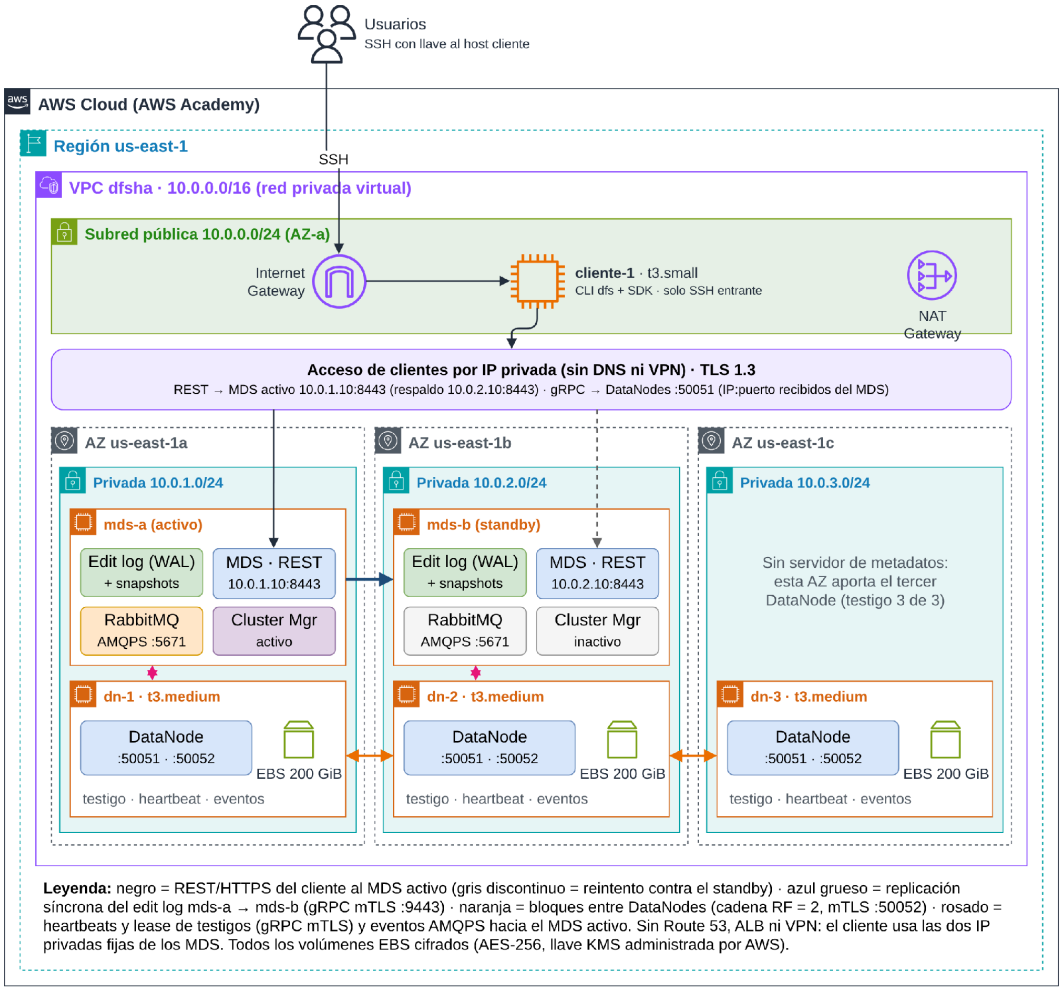


Figura 2. Despliegue de DFSha en AWS (íconos oficiales de AWS, modelado en draw.io).

Tabla 3. Componentes, responsabilidades y estado.

Componente	Plano	Responsabilidades	Estado y tecnología
Client SDK + CLI `dfs`	Cliente	Partir y reensamblar bloques, calcular checksums, E/S paralela (P = 4), caché de ubicaciones, reintentos, failover entre réplicas y entre MDS, renovación de leases.	Sin estado durable. Java 21, Picocli, grpc-java.
MDS activo (mds-a)	Control	Autenticación, autorización y auditoría; espacio de nombres; asignación de bloques y block tokens; locks y leases; commit atómico; Cluster Manager.	Estado en memoria + edit log (WAL) + snapshots en EBS cifrado. Spring Boot, HTTPS 8443.
MDS standby (mds-b)	Control	Recibe y persiste cada entrada del edit log antes de que el activo responda; mantiene el estado en memoria al día; se promueve si obtiene el lease de los testigos.	Mismo software y almacenamiento; gRPC mTLS 9443.

Componente	Plano	Responsabilidades	Estado y tecnología
Cluster Manager (CM)	Control	Detector de fallas, re-replicación, recolección de bloques (GC), reconciliación de block reports.	Módulo del MDS; solo corre en el activo.
RabbitMQ local	Control (asíncrono)	Eventos de los DataNodes (bloque recibido, borrado o corrupto; block reports) y colas de comandos por nodo.	Un broker en cada MDS; se usa el del activo. AMQPS 5671.
DataNode ×3	Datos	Guardar y servir bloques (`.part` → `.blk`), replicar en cadena, scrubbing SHA-256, heartbeats y voto de testigo, ejecutar comandos.	EBS gp3 cifrado. gRPC 50051 (clientes, TLS) y 50052 (nodos, mTLS).

Plano de control: decide qué archivos existen, quién accede, qué bloques forman cada versión y dónde están. **Plano de datos:** mueve los bytes con streams gRPC. Como los datos nunca pasan por el MDS, el rendimiento agregado escala con los DataNodes (RNF1, RNF5).

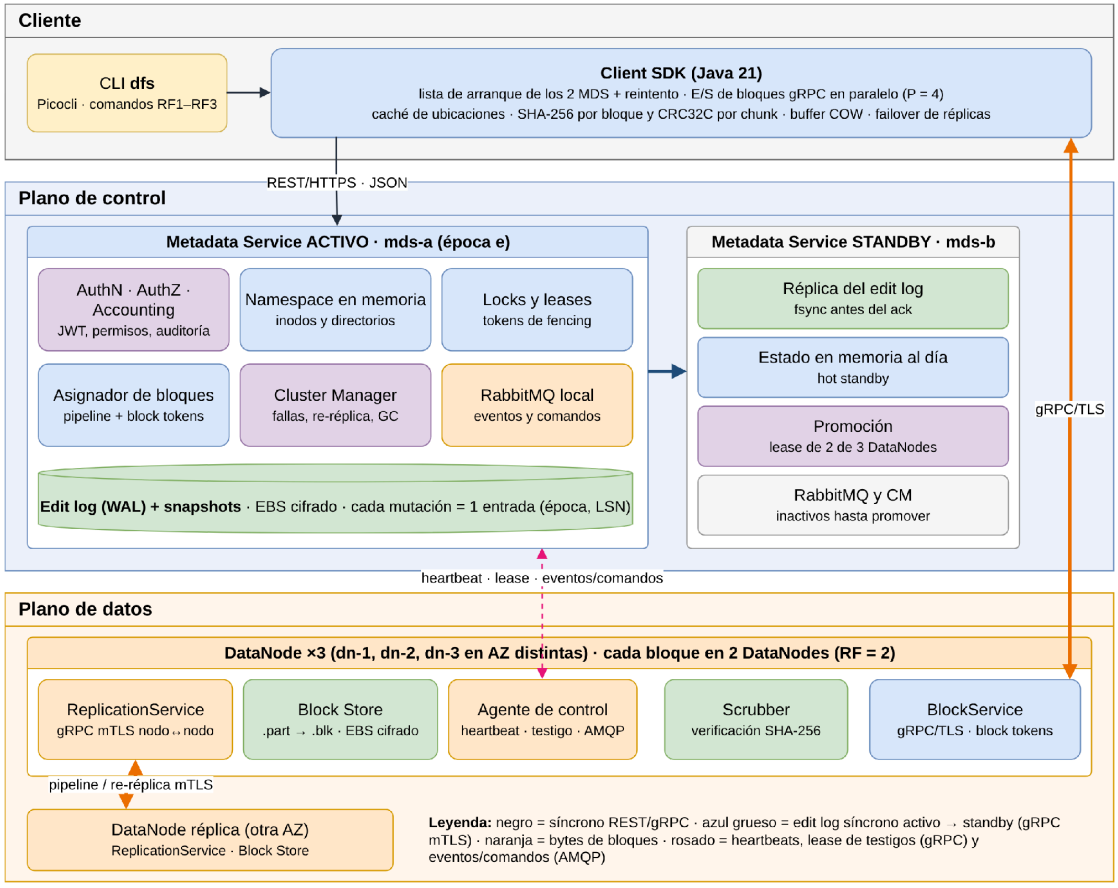


Figura 3. Componentes de DFSha, separados en plano de control y plano de datos.

3.1 Despliegue en AWS

El despliegue (Figura 2) usa una VPC 10.0.0.0/16 en tres AZ con seis instancias. `cliente-1` (t3.small) está en la subred pública y solo admite SSH. `mds-a` (AZ-a) y `mds-b` (AZ-b) tienen IP privadas fijas (10.0.1.10 y 10.0.2.10). `dn-1`, `dn-2` y `dn-3` van uno por AZ (t3.medium, EBS gp3 de 200 GiB), así que las dos réplicas de un bloque siempre quedan en AZ distintas y cada nodo guarda ~2/3 de los bloques. Los DataNodes son también los **testigos** que arbitran qué MDS está activo. No hay Route 53, ALB ni VPN. Las instancias privadas no tienen IP pública y salen por el NAT Gateway; los grupos de seguridad solo abren los puertos de la Tabla 3.

4. Distribución en bloques, metadatos y réplicas

4.1 Particionamiento

Cada archivo se divide en bloques de **B = 128 MiB**, salvo el último, que ocupa solo su tamaño real. Cada bloque tiene un identificador UUIDv7 (`blk_…`) y un **generation stamp** que aumenta si el bloque debe reenviarse. Los bloques son **inmutables**: toda modificación crea un bloque nuevo (copy-on-write), así que una réplica nunca tiene un contenido distinto bajo el mismo `(blockId, gen)`. La transferencia usa chunks de 1 MiB con CRC32C, y cada bloque lleva su SHA-256. Un archivo de 10 GiB tiene 80 bloques y 160 réplicas (~53 por DataNode). Se eligió 128 MiB, como HDFS [2], porque los metadatos viven en la memoria del MDS (~150 B por bloque) y porque bloques grandes favorecen el throughput secuencial. El costo: una modificación parcial reescribe el bloque completo y los archivos pequeños aprovechan poco el paralelismo.

4.2 Política de ubicación de réplicas (RF = 2)

1. Candidatos: DataNodes en estado ALIVE con espacio libre > 2B.
2. Réplica 1: el mejor de dos candidatos al azar según conexiones activas y % de disco usado (power of two choices).
3. Réplica 2: el mejor candidato restante, que por construcción está en **otra AZ**. El pipeline es R1 → R2: un solo salto entre nodos.
4. El nodo inicial rota entre bloques consecutivos, de modo que los bloques de un archivo se reparten entre los tres nodos (RNF4) y la lectura paralela usa los tres.

4.3 Metadatos

El MDS mantiene el estado en memoria (como el NameNode de HDFS) y lo hace durable con un **edit log** de solo anexado, más un snapshot cada 10 minutos o cada 100 000 entradas; al reiniciar carga el último snapshot y reaplica el log. Cada entrada lleva `(época, LSN, operación, CRC)`.

Tabla 4. Estructuras de metadatos del MDS.

Estructura	Contenido y uso
Directorios	`(inodoPadre, nombre) → inodeId`; resuelve rutas y `ls`; `mv` cambia dos entradas en una sola operación del log.
Inodos	`{owner, mode, size, version, fileSha256, ctime, mtime, blocks:[{idx, blockId, gen, len, sha256}]}`; versión confirmada, solo cambia en un commit.
Mapa de bloques	`blockId → {gen, len, sha256, replicas[], refCount}`; las réplicas son estado blando reconstruido con block reports; refCount sirve a COW, `cp` y GC.
Nodos, locks y sesiones	`nodeId → {ip:puerto, az, capacidad, estado, último heartbeat, llave pública}`; locks `{rango, modo, dueño, vence, fencingToken}`; handles abiertos por versión.
Usuarios y auditoría	Hash Argon2id, rol, cuota y uso; registro de auditoría de solo anexado (accounting).

Capacidad: con ~300 B por inodo y ~150 B por bloque, un millón de archivos de 1 GiB ocupa ~1,5 GB de RAM; la memoria del MDS activo es el límite de escala de metadatos.

4.4 Localización y reconstrucción

`open()` devuelve la versión confirmada con la lista ordenada de bloques. Para cada bloque incluye sus réplicas vivas (IP:puerto y AZ, la más cercana primero) y un block token. El SDK pide hasta 4 bloques en paralelo y los escribe directamente en su offset `idx × B` del archivo local, sin acumularlos en memoria. Verifica el CRC32C de cada chunk y el SHA-256 de cada bloque, y al final compara el SHA-256 del archivo con `fileSha256`. Si una réplica falla, el SDK usa la otra y la reporta al MDS.

5. Protocolos y APIs

Tabla 5. Protocolo por canal y justificación.

Canal	Protocolo	Justificación
Cliente ↔ MDS	REST/JSON, HTTPS (TLS 1.3) :8443	Operaciones pequeñas y depurables; errores RFC 9457 (problem+json) [12].
Cliente ↔ DataNode	gRPC/HTTP 2 + TLS :50051	Streaming con control de flujo, deadlines y contratos protobuf para bloques de 128 MiB.
DataNode ↔ DataNode	gRPC + mTLS :50052	Replicación en cadena y re-replicación autenticada [4].
MDS ↔ MDS, DataNode ↔ MDS	gRPC + mTLS :9443 / :50052	Edit log síncrono, heartbeats, lease de testigos y block reports: llamadas con respuesta y plazo.
DataNode ↔ MDS (asíncrono)	AMQP 0-9-1 + TLS :5671	Eventos y comandos con confirmación del publicador y ack manual [11]; se prefiere a MQTT (plugin con QoS limitado) y a Kafka (logs de alto volumen).

5.1 API REST del MDS (https://10.0.1.10:8443/v1, respaldo en 10.0.2.10)

Tabla 6. Endpoints. Todos requieren `Authorization: Bearer <token>` salvo `login`. El standby responde 503 NOT_ACTIVE con la dirección del activo.

Método y ruta	Descripción	Errores típicos
`POST /auth/login`, `/auth/refresh`, `/auth/logout`	Emite un token de acceso (15 min) y un refresh token (8 h); `logout` los revoca.	401 INVALID_CREDENTIALS, 429 TOO_MANY_ATTEMPTS
`GET /fs/ls?path=&cursor=`, `GET /fs/stat?path=`	Lista paginada y atributos.	403 FORBIDDEN, 404 NOT_FOUND
`POST /fs/mkdir`, `DELETE /fs/rmdir?recursive=`	Crea o borra un directorio.	409 ALREADY_EXISTS, 409 DIR_NOT_EMPTY
`DELETE /fs/files?path=`, `POST /fs/move`, `POST /fs/copy`	Borrado lógico; renombrado atómico; copia COW.	404, 409, 507 QUOTA_EXCEEDED
`POST /files:open`	Abre en modo `r|w|a|rw` y devuelve handle, versión, bloques, réplicas, token y lease.	404, 409 LOCK_CONFLICT

Método y ruta	Descripción	Errores típicos
`POST /files/{h}:allocate`, `:recover-block`	Asigna bloques con su pipeline y writeToken; regenera el pipeline con gen + 1.	503 INSUFFICIENT_NODES, 410 HANDLE_EXPIRED
`POST /files/{h}:commit`, `:close`, `:keepalive`	Publica la versión de forma atómica; cierra; renueva el lease.	409 VERSION_CONFLICT, 409 STALE_FENCING_TOKEN, 422 BLOCK_NOT_DURABLE
`POST /locks`, `DELETE /locks/{id}`	`{path, mode, range, ttlMs, waitMs}` devuelve `{lockId, fencingToken, expiresAt}`.	409 LOCK_CONFLICT
`POST /reports/replica`	El cliente reporta una réplica inalcanzable o corrupta.	—
Cualquiera en el standby	Redirige al activo.	503 NOT_ACTIVE `{activeEndpoint}`

Ejemplo de escritura y de respuestas de error:

```
POST /v1/files:open   {"path":"/home/jose/ventas.csv","mode":"w","create":true}
200 {"handle":"h_9f2c","baseVersion":6,"blockSize":134217728,"lease":{"fencingToken":"3:88120","ttlMs":30000}}
POST /v1/files/h_9f2c:allocate {"firstIdx":0,"count":2}
200 {"writeToken":"eyJhbGci…","blocks":[{"idx":0,"blockId":"blk_01JBX4","gen":1,
     "pipeline":["10.0.1.21:50051","10.0.2.37:50051"]}, …]}
POST /v1/files/h_9f2c:commit {"fencingToken":"3:88120","size":201326592,"fileSha256":"9b1d…",
     "blocks":[{"idx":0,"blockId":"blk_01JBX4","gen":1,"sha256":"a3f0…",
               "receipts":[{"node":"dn-1","sig":"…"},{"node":"dn-2","sig":"…"}]}, …]}
200 {"version":7}
409 {"type":"/errors/stale-fencing-token","code":"STALE_FENCING_TOKEN","detail":"reemplazado por 4:12"}
503 {"code":"NOT_ACTIVE","activeEndpoint":"https://10.0.2.10:8443"}
```

5.2 Contratos gRPC (resumen protobuf)

```
service BlockService {       // DataNode :50051, TLS + block token
  rpc WriteBlock(stream WriteReq) returns (WriteResp);   // header → chunks 1 MiB + CRC32C → trailer SHA-256
  rpc ReadBlock(ReadReq) returns (stream Chunk); }       // rango [offset, offset+length)
service ReplicationService { rpc ReplicateBlock(stream WriteReq) returns (WriteResp); } // DN :50052, mTLS
service WitnessService {     // DataNode :50052, mTLS, llamado por los MDS
  rpc RenewLease(LeaseReq) returns (LeaseResp);          // {epoch, mdsId, lastLsn, standbyInSync}
  rpc Vote(VoteReq) returns (LeaseResp); }               // {epoch+1, mdsId, lastLsn}
service ControlService {     // MDS :9443, mTLS, llamado por los DataNodes
  rpc Register(NodeInfo) returns (RegisterResp);  rpc Heartbeat(Hb) returns (HbResp);
  rpc BlockReport(stream BlockRef) returns (Ack); }
service MetaReplication {    // MDS standby :9443, mTLS
  rpc Append(stream LogEntry) returns (stream AppendAck);  rpc FetchSnapshot(SnapReq) returns (stream SnapChunk); }
```

Errores gRPC. `UNAUTHENTICATED`/`PERMISSION_DENIED`: token ausente o bloque fuera de su alcance. `NOT_FOUND`/`FAILED_PRECONDITION`: bloque o generación inexistente u obsoleta. `DATA_LOSS`: checksum incorrecto (con el offset). `ABORTED`: falló el siguiente nodo de la cadena (`failedNodes`). `RESOURCE_EXHAUSTED`: disco lleno. `UNAVAILABLE`/`DEADLINE_EXCEEDED`: nodo caído o tiempo agotado (5 s por chunk). **Mensajería:** el exchange `dfs.events` (topic) lleva `block.received|deleted|corrupt.<id>`; el exchange `dfs.commands` (direct) entrega a la cola `dn.<id>.commands` mensajes como `{"commandId":"c-7f3a","type":"REPLICATE_BLOCK","blockId":"blk_01JBX4","gen":1,"target":"10.0.3.14:50052"}` o `DELETE_BLOCK`. Los comandos son idempotentes por `commandId`.

5.3 Secuencias de carga/escritura, lectura y borrado

Lectura. `open(r)` devuelve un snapshot de versión con las réplicas de cada bloque y un block token; el SDK lee hasta 4 bloques en paralelo con `ReadBlock`, pasa a la otra réplica ante `UNAVAILABLE` o checksum incorrecto, reporta la réplica fallida (`POST /reports/replica`), verifica los SHA-256 y cierra con `close()`. **Carga y escritura:** Figura 4.}

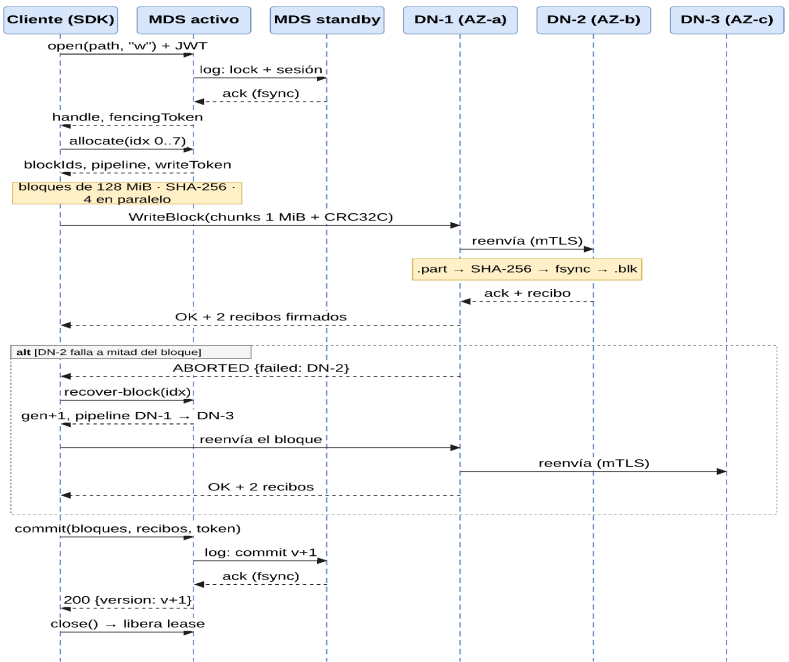


Figura 4. Carga y escritura (put/write): lease, asignación, replicación en cadena (RF = 2), recuperación de pipeline con el tercer nodo y commit replicado en el standby.

Borrado. `rm` borra la entrada del directorio y decrementa `refCount` de los bloques en una sola operación del log; el archivo deja de verse de inmediato (204). Los bloques que llegan a `refCount = 0` esperan 10 minutos, o más si quedan lectores sobre esa versión. Luego el CM envía `DELETE_BLOCK` a la cola de cada réplica, el DataNode borra el `.blk` y confirma con `block.deleted`. Así el espacio se libera de forma verificable y los datos borrados no quedan legibles indefinidamente.

6. Semántica de read() y write()

Tabla 7. Semántica declarada de RF3 (no se pretende POSIX completo).

Aspecto	Decisión
Granularidad	La API trabaja en bytes: `read(fd, off, len)` y `write(fd, off, data)`. La persistencia y la replicación trabajan por bloque de 128 MiB con copy-on-write.
Creación y reemplazo	`open(p, "w")` crea el archivo o escribe una versión nueva completa; en el commit se reemplaza el mapa de bloques de forma atómica y los bloques viejos pasan a GC diferido. Con `exclusive=true`, falla con 409 si la ruta ya existe.
Anexado	`open(p, "a")`: el último bloque parcial se reescribe como bloque nuevo y se agregan bloques. Requiere el lock del archivo, porque cambia el tamaño.
Modificación parcial	`open(p, "rw")` + `lock(rango)` + `write(off, …)`: el SDK lee, modifica y reescribe solo los bloques tocados (hasta 128 MiB cada uno) y el commit los reemplaza con CAS por posición. Los demás bloques se reutilizan por referencia.
Visibilidad	Close-to-open [5]: los cambios se publican en `close()` o `flush()` (cada `flush()` crea una versión intermedia). Los lectores abiertos conservan su snapshot; un `open()` posterior ve la versión nueva.
Durabilidad	Que `write()` retorne no implica durabilidad: los datos van a un buffer local. La durabilidad llega con `flush()` o `close()` exitosos (G1).
Concurrencia	Lectores ilimitados, concurrentes con escritores. Puede haber varios escritores en rangos disjuntos; en un mismo rango, uno a la vez (sección 7).
No soportado	`truncate` arbitrario, `mmap`, enlaces, compartir entre usuarios.

7. Consistencia, exclusión mutua, integridad y recuperación

7.1 Consistencia por versiones y replicación de metadatos

Cada archivo tiene una secuencia de versiones; cada versión es una lista inmutable de bloques inmutables. El MDS activo aplica las mutaciones **en un único orden** (el del edit log). Un commit se valida (`versión base`, fencing token y recibos) y se registra como **una** entrada. La entrada se envía al standby, que hace fsync y confirma (replicación primario-respaldo síncrona [6]); solo entonces el activo la aplica en memoria y responde 200. Como un bloque nunca cambia de contenido, las réplicas de datos no necesitan protocolo de consistencia propio: el problema queda reducido a mantener ordenados los metadatos.

Si el standby no responde en 2 s, el activo lo marca OUT_OF_SYNC, sigue operando solo (modo degradado, con alarma) y se lo informa a los testigos. Cuando el standby vuelve, se resincroniza con `FetchSnapshot` y la cola del log antes de volver a IN_SYNC. Un standby OUT_OF_SYNC **no puede** promoverse, porque perdería commits confirmados: en ese caso se prefiere la consistencia a la disponibilidad.

7.2 Arbitraje del MDS activo sin consenso externo

Los tres DataNodes actúan como **testigos** del lease del MDS activo, con la misma idea de los leases de Gray y Cheriton [7] y de la regla de mayoría [8]; es un esquema análogo al par activo/standby de HDFS HA [3], pero sin servicio externo de coordinación:

1. El activo (época e) llama `RenewLease(e, lastLsn, standbyInSync)` cada 2 s a los 3 DataNodes. Cada testigo concede un lease de 10 s si no conoce una época mayor ni tiene otro lease vigente. El activo solo atiende peticiones mientras tiene ≥ 2 leases vigentes, con un margen de 2 s; si no logra renovarlos, se degrada a standby por sí mismo.
2. Si el standby pasa 6 s sin recibir el edit log ni pings del activo, espera a que expire el lease y pide `Vote(e+1, lastLsn)`. Un testigo vota solo si el lease anterior venció, si el último estado que vio indicaba `standbyInSync = true` y si `lastLsn` del standby es ≥ al último LSN confirmado que conoce.
3. Con 2 de 3 votos, el standby pasa a activo con época e+1: arranca su CM y su broker, pide block reports completos y extiende todos los locks un TTL completo para que sus dueños los renueven. Si el antiguo activo vuelve, ve una época mayor, se convierte en standby y se resincroniza.

Como una mayoría de 2 de 3 no puede formarse dos veces en la misma época, **nunca hay dos activos**. Un MDS aislado de los testigos deja de servir antes de que el otro sea promovido (Figura 5).

7.3 Exclusión mutua: lock() con leases y fencing

`lock(path|fd, mode, range, ttl, wait)` es **obligatorio para escribir**: el MDS rechaza un commit si no hay un lock vigente que cubra los bloques tocados, y `open("w"|"a")` lo adquiere implícitamente sobre todo el archivo. Los modos son SHARED y

EXCLUSIVE, sobre el archivo o un rango de bloques; las lecturas no se bloquean. Cada lock vive en la tabla de locks del MDS (replicada por el edit log) con un TTL de 30 s que el SDK renueva cada 10 s; si el cliente muere, el lock expira solo. Con `waitMs`, las solicitudes esperan en una cola FIFO. Cada concesión entrega un **fencing token** `época:secuencia`, monótono incluso entre conmutaciones. El commit compara el token, así que un cliente pausado cuyo lock expiró no puede publicar nada [10].

7.4 Integridad y escrituras interrumpidas

La integridad se verifica en tres niveles: CRC32C por chunk, SHA-256 por bloque (lo calcula el cliente, lo verifica el DataNode antes de confirmar y lo vuelve a comprobar el lector) y SHA-256 del archivo completo. Un scrubber en cada DataNode recalcula periódicamente los hashes y reporta `block.corrupt`. La integridad detecta corrupción, pero no coordina: eso lo hacen los locks y el CAS. **Escrituras interrumpidas:** el DataNode escribe en `.part` y solo emite su recibo firmado tras verificar el hash, hacer fsync y renombrar a `.blk`. Si falla un nodo, el cliente pide `recover-block` (gen + 1) y reenvía el bloque. Si falla el cliente, el lease vence en 30 s, la sesión se aborta y los bloques sin referencia van a GC tras 1 h. Si falla el MDS durante el commit, la entrada o está en el log de ambos MDS o no está en ninguno, y el SDK reintenta el commit (idempotente por `handle`).

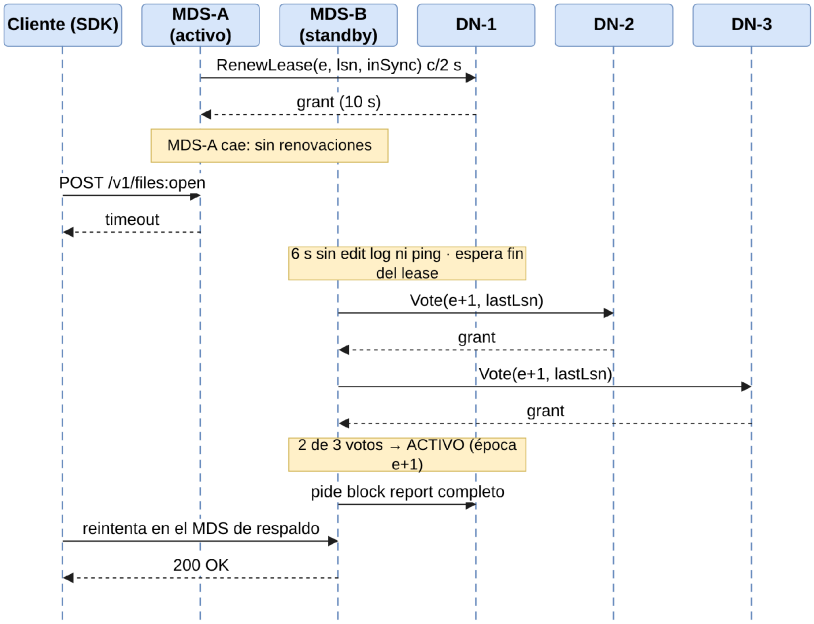


Figura 5. Conmutación del MDS: el activo deja de renovar su lease y el standby obtiene 2 de 3 votos de los testigos.

8. Tolerancia a fallos y disponibilidad

Cada DataNode envía un heartbeat gRPC cada 3 s a ambos MDS. El CM del activo marca un nodo como **SUSPECT** a los 10 s (deja de recibir pipelines nuevos) y como **DEAD** a los 30 s (deja de aparecer en las ubicaciones de lectura). Los heartbeats no pasan por RabbitMQ, así que una caída del broker no se confunde con la caída de los nodos.

Tabla 8. Fallas, detección y recuperación.

Falla	Detección	Efecto y recuperación
DataNode caído	Heartbeats (10/30 s); errores gRPC	Lectura: la otra réplica (timeout 5 s). Escritura: `recover-block` con el tercer nodo. Tras 2 min, el CM copia cada bloque afectado desde su réplica superviviente al otro nodo vivo (50 MB/s).
Réplica corrupta	CRC/SHA-256 o scrubber	El cliente usa la otra réplica y la reporta; el CM la marca CORRUPT, copia la sana y la borra.
MDS activo caído	Fin del lease (10 s)	~15 s de errores reintentables; el standby IN_SYNC se promueve con 2 de 3 votos (Figura 5) y el SDK lo usa.
MDS standby caído	Append sin ack (2 s)	Sin efecto visible; el activo queda degradado. Al volver: `FetchSnapshot` + cola del log → IN_SYNC.
Broker caído	Conexión AMQP rota	Sin efecto en E/S; los DataNodes guardan eventos en una bandeja local y un block report completo reconcilia.
Cliente caído	Lease de escritura (30 s)	El archivo conserva su versión; lock liberado, sesión abortada, GC de huérfanos.
Pérdida de una AZ	Todo lo anterior	Siguen lecturas, escrituras (2 DataNodes) y metadatos (un MDS + 2 testigos); luego se re-replica.

9. Seguridad: cifrado y AAA con TLS y permisos

Autenticación: usuarios con contraseña de ≥ 12 caracteres guardada con Argon2id [13]; el MDS emite un JWT EdDSA de 15 min y un refresh token revocable de 8 h [14]; 429 tras 5 fallos por cuenta o 10 por IP en 15 min. Los servidores se autentican con certificados X.509 de una CA interna (mTLS), con el rol (`metadata` o `datanode`) en el campo OU.

Autorización (permisos): cada inodo tiene dueño y bits de modo; el MDS exige `owner == sub` (o rol ADMIN), canoniza rutas, limita a `/home/<sub>` y aplica cuotas. Los DataNodes solo aceptan **block tokens** firmados por el MDS (`blk[]`, `ops = r|w`, 10 min), y entre servidores cada RPC valida el rol del certificado (solo `metadata` puede llamar `RenewLease`).

Accounting: registro de auditoría de solo anexado (quién, qué, cuándo, IP, resultado, `traceId`) replicado con el edit log, más el uso por usuario (bytes almacenados y transferidos).

Cifrado: TLS 1.3 en todos los canales, con mTLS entre servidores [15]. En reposo, todos los volúmenes EBS (bloques, edit log, auditoría) van cifrados con AES-256 y llave KMS administrada por AWS [16]. Los grupos de seguridad solo abren los puertos de la sección 5 y SSH al host cliente.

10. Métricas y experimentos previstos

Método: generador de carga propio (`dfs-bench`, Java, HdrHistogram); 5 repeticiones tras calentamiento, con la media y un IC del 95 %. Las métricas del servidor se exponen con Micrometer; la latencia se inyecta con `tc netem` y las fallas con `kill -9`, `iptables` o la parada de instancias, todo con scripts reproducibles.

Tabla 9. Plan experimental.

Id	Objetivo	Variables	Métricas	Hipótesis
E1	Latencia de metadatos	`ls`, `stat`, `open`, `commit`; 1–64 clientes	p50/p95/p99, ops/s	`commit` ≈ `stat` + 1 RTT + fsync (réplica síncrona); p95 < 30 ms con ≤ 16 clientes.
E2	Throughput	Tamaño 256 MiB–8 GiB; B = 64/128/256 MiB; P = 1–8	MB/s de put y get	get escala con P hasta el límite de red de 3 nodos.
E3	Costo de la replicación	RF = 1 frente a RF = 2	Latencia y MB/s de put	RF = 2 da ≈ 0,85–0,95× el throughput de RF = 1 (la cadena es de un solo salto).
E4	Efecto de la concurrencia	2–16 escritores: mismo bloque, bloques disjuntos, archivos distintos	Espera por lock, % de 409, throughput	El mismo bloque se serializa; los rangos disjuntos escalan.
E5	Fallas de DataNode	kill durante get/put; réplica desconectada	Tiempo de failover, % de éxito, MTTR	0 lecturas fallidas; MTTR ≈ GB afectados / 50 MB/s.
E6	Conmutación del MDS	kill del activo; partición del activo (`iptables`); standby caído	Segundos sin servicio, commits perdidos, número de activos simultáneos	~15 s; 0 commits confirmados perdidos; nunca dos activos.
E7	Escalabilidad	Clientes 1 → 32; DataNodes 2 → 3	Throughput agregado	Casi lineal en datos; el MDS no limita hasta miles de ops/s.
E8	Corrección y seguridad	Hash de ida y vuelta; accesos cruzados; captura de tráfico; `describe-volumes`	% de hashes iguales, % de accesos denegados	100 % y 100 %; tráfico ilegible; volúmenes con `Encrypted=true`.

11. Escenarios obligatorios de razonamiento

11.1 Dos procesos escriben el mismo bloque simultáneamente

Ambos abren el archivo en modo `rw` y piden `lock(fd, [k,k], EXCLUSIVE)`. El MDS activo procesa las solicitudes en el orden de su log: la primera (P1) recibe el lock con el token `e:s` y la segunda (P2) recibe 409 LOCK_CONFLICT, o espera en la cola si pidió `waitMs`. P1 escribe un bloque **nuevo** `k'` y confirma la versión v+1. Cuando P1 libera, P2 obtiene el lock con un token mayor y la versión vigente. Si P2 tenía datos preparados sobre v, su commit falla por CAS (409 VERSION_CONFLICT) y debe releer y reaplicar. Si P1 se queda pausado más allá de su TTL, su lock expira, P2 lo obtiene y el commit tardío de P1 recibe 409 STALE_FENCING_TOKEN; sus bloques quedan huérfanos y van a GC. Vale también si el MDS conmuta entre medio: la tabla de locks está en el edit log y la época forma parte del token. **Resultado:** escrituras serializadas, sin pérdidas silenciosas (G4).

11.2 Una réplica está desconectada durante una escritura

Si el nodo ya estaba SUSPECT, el MDS no lo incluye en el pipeline. Si se desconecta a mitad del bloque, el pipeline aborta y el cliente obtiene gen + 1 con el tercer DataNode, así que el commit se hace igualmente con 2 réplicas. Si solo quedan 2 nodos vivos y uno de ellos se desconecta, la escritura no puede alcanzar W = 2: el cliente recibe 503 INSUFFICIENT_NODES y el archivo conserva su versión anterior, porque se prefiere no confirmar con una sola copia. **Lectores:** los que abrieron antes leen v en las réplicas existentes; los que abren después del commit leen v+1. Ninguno puede recibir datos viejos del nodo desconectado, porque v+1 referencia `blockId` nuevos que ese nodo no tiene. **Recuperación:** al reconectarse, el nodo envía un block report completo; el CM elimina sus `.part` y bloques sin referencia, y re-replica los bloques que quedaron con una sola réplica.

11.3 Un nodo falla después de recibir parte de un bloque

El DataNode solo renombra `blk_x_g.part` a `.blk` cuando tiene la longitud completa, el SHA-256 coincide y hizo `fsync`; solo entonces firma su recibo. Si cae antes, no hay recibo y el cliente no reúne los 2 recibos necesarios, así que pide `recover-block` y reenvía el bloque entero a un pipeline nuevo. **No se publica un archivo incompleto** porque el commit exige que estén todos los índices 0…n−1, cada uno con 2 recibos válidos de la generación vigente, y que la suma de longitudes sea igual a `size`. Si algo falta, la respuesta es 422 BLOCK_NOT_DURABLE y el inodo sigue en la versión anterior. Los `.part` se limpian al reiniciar el nodo o, como huérfanos, después de 1 h.

11.4 El cliente desconoce la ubicación física de los bloques

Sin DNS, la configuración del cliente contiene solo la **lista de arranque** de los dos MDS (10.0.1.10 y 10.0.2.10, puerto 8443) y el certificado de la CA, nunca direcciones de DataNodes. Si contacta al standby, este responde 503 NOT_ACTIVE con la dirección del activo; si un MDS no responde, el SDK prueba el otro. Los DataNodes **se registran solos** al arrancar (`Register` con la IP privada, el puerto y la AZ obtenidos de los metadatos de la instancia), así que agregar o reemplazar un nodo no requiere cambiar nada en los clientes. En `open()` el MDS devuelve, para cada bloque, las IP:puerto de sus réplicas vivas. El SDK las cachea 60 s y, ante UNAVAILABLE o NOT_FOUND, prueba la otra réplica y pide ubicaciones nuevas (transparencia de acceso y localización, RNF7).

12. Decisiones, compromisos y limitaciones

Tabla 10. Registro de decisiones arquitectónicas.

Decisión	Alternativa descartada	Justificación	Costo aceptado
2 MDS activo/standby con edit log síncrono y testigos	NFS compartido; consenso (Raft/etcd, fuera del alcance)	Sin punto único de falla ni split-brain, con solo dos servidores de metadatos.	Conmutación de ~15 s; standby ocioso; un solo MDS activo limita la escala.
Metadatos en memoria + log + snapshots	Base de datos externa	Latencia baja y recuperación determinista, como HDFS [2][3].	La RAM limita el número de archivos.
Bloques de 128 MiB inmutables (COW)	Bloques pequeños; escritura en sitio	Pocos metadatos, throughput secuencial y consistencia de réplicas trivial.	Modificar 1 byte reescribe hasta 128 MiB.
RF = 2, W = 2, 3 DataNodes en 3 AZ	RF = 3	Almacenamiento 2×; tolera la pérdida de un nodo o una AZ.	Dos fallas simultáneas pueden perder bloques.
Broker local en cada MDS	Clúster de RabbitMQ	Sin clúster de broker; no está en la ruta crítica.	Eventos en tránsito al conmutar; se reconcilian con block reports.
Lista de arranque de IP (sin DNS)	DNS o balanceador	Cumple el alcance; los DataNodes se descubren dinámicamente.	Cambiar la IP de un MDS exige actualizar la configuración del cliente.

Limitaciones declaradas: sin compartir archivos entre usuarios; RF3 acotado a la sección 6; los clientes deben ejecutarse dentro de la VPC; si el standby está desincronizado cuando cae el activo, se requiere intervención manual; los archivos pequeños aprovechan poco el paralelismo; no hay cifrado del lado del cliente. Toda simplificación adicional se acordará con el profesor antes de implementar.

Referencias

[1] S. Ghemawat, H. Gobioff y S.-T. Leung, "The Google File System," en Proc. 19th ACM SOSP, 2003.
[2] K. Shvachko, H. Kuang, S. Radia y R. Chansler, "The Hadoop Distributed File System," IEEE MSST, 2010.
[3] Apache Hadoop, "HDFS High Availability" y "HDFS Architecture," hadoop.apache.org/docs.
[4] R. van Renesse y F. B. Schneider, "Chain Replication for Supporting High Throughput and Availability," OSDI, 2004.
[5] J. H. Howard et al., "Scale and Performance in a Distributed File System," ACM Trans. Comput. Syst., 6(1), 1988.
[6] N. Budhiraja, K. Marzullo, F. B. Schneider y S. Toueg, "The Primary-Backup Approach," en Distributed Systems (2.ª ed.), 1993.
[7] C. Gray y D. Cheriton, "Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency," SOSP, 1989.
[8] M. van Steen y A. S. Tanenbaum, Distributed Systems, 4.ª ed., distributed-systems.net, 2023.
[9] S. Gilbert y N. Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services," ACM SIGACT News, 2002.
[10] M. Kleppmann, Designing Data-Intensive Applications. O'Reilly, 2017, caps. 5, 8 y 9.
[11] RabbitMQ, "Consumer Acknowledgements and Publisher Confirms," rabbitmq.com/docs.
[12] IETF RFC 9457 "Problem Details for HTTP APIs" (2023); gRPC Authors, "Status codes," grpc.io.
[13] OWASP, "Password Storage Cheat Sheet," cheatsheetseries.owasp.org; NIST SP 800-63B, 2017.
[14] IETF RFC 7519, "JSON Web Token," 2015; RFC 8037, "EdDSA in JOSE," 2017.
[15] IETF RFC 8446, "The Transport Layer Security (TLS) Protocol Version 1.3," 2018.
[16] AWS, "Amazon EBS encryption" y "Well-Architected Framework: Reliability Pillar," docs.aws.amazon.com.